

```
1 import sys
2 input = sys.stdin.readline
3
4 class PQMinList:
5
6     def __init__(self):
7         self._data = []
8
9     def push(self, element: int) -> None:
10         self._data.append(element)
11
12     def pop(self) -> int:
13         if not self._data:
14             raise IndexError("pop from empty PQ")
15         e = min(self._data)
16         self._data.remove(e)
17         return e
18
19     def peek(self) -> int:
20         if not self._data:
21             raise IndexError("peek from empty PQ")
22         return min(self._data)
23
24     def size(self) -> int:
25         return len(self._data)
26
27     def copy(self) -> "PQMinList":
28         mh = PQMinList()
29         mh._data = self._data[:] # shallow copy is sufficient as we are only working with non-mutable data (
    int).
30         return mh
31
32
33 class PQMinHeap:
34
```

```
35     def __init__(self):
36         self._data = []
37
38     def _parent(self, i: int) -> int:
39         return (i - 1) // 2
40
41     def _left(self, i: int) -> int:
42         return 2 * i + 1
43
44     def _right(self, i: int) -> int:
45         return 2 * i + 2
46
47     def _heapify_up(self, i: int) -> None:
48         """Bubble the element at index i up until the heap property is restored."""
49         while i > 0:
50             parent = self._parent(i)
51             if self._data[i] < self._data[parent]:
52                 self._data[i], self._data[parent] = self._data[parent], self._data[i]
53                 i = parent
54             else:
55                 break
56
57     def _heapify_down(self, i: int) -> None:
58         """Sink the element at index i down until the heap property is restored."""
59         n = len(self._data)
60         while True:
61             smallest = i
62             left = self._left(i)
63             right = self._right(i)
64             if left < n and self._data[left] < self._data[smallest]:
65                 smallest = left
66             if right < n and self._data[right] < self._data[smallest]:
67                 smallest = right
68             if smallest == i:
69                 break
```

```
70         self._data[i], self._data[smallest] = self._data[smallest], self._data[i]
71         i = smallest
72
73     def push(self, element: int) -> None:
74         """Insert a new element in O(log n)."""
75         self._data.append(element)
76         self._heapify_up(len(self._data) - 1)
77
78     def pop(self) -> int:
79         """Remove and return the minimum element in O(log n)."""
80         if not self._data:
81             raise IndexError("pop from empty PQ")
82
83         # Swap root with the last element, shrink, then restore heap property.
84         self._data[0], self._data[-1] = self._data[-1], self._data[0]
85         minimum = self._data.pop()
86         if self._data:
87             self._heapify_down(0)
88         return minimum
89
90     def peek(self) -> int:
91         """Return the minimum element without removing it in O(1)."""
92         if not self._data:
93             raise IndexError("peek from empty PQ")
94         return self._data[0]
95
96     def size(self) -> int:
97         return len(self._data)
98
99     def copy(self) -> "PQMinHeap":
100         mh = PQMinHeap()
101         mh._data = self._data[:]
102         return mh
103
104
```

```

105 def heaps():
106     heaps = [PQMinHeap() for _ in range(1001)]
107
108     n = int(input())
109     for _ in range(n):
110         inputs = input().split()
111         id = int(inputs[0])
112         command = inputs[1]
113         value = int(inputs[2]) if len(inputs) > 2 else 0
114         if command == "a":
115             heaps[id] = heaps[value].copy()
116         elif command == "+":
117             heaps[id].push(value)
118         elif command == "-":
119             heaps[id].pop()
120         elif command == "p":
121             print(heaps[id].peek())
122         elif command == "s":
123             print(heaps[id].size())
124         else:
125             print("Invalid command")
126
127
128 def voff():
129     # We use the heap to keep track of the dogs we know exist.
130     # The heap priority is the time, t, a given dog can next bark.
131     mh = PQMinHeap()
132     n, k = map(int, input().split())
133     mh.push(0) # Assume a single dog, ready to bark!
134     ts = list(map(int, input().split()))
135     for t in ts:
136         if t >= mh.peek():
137             mh.pop() # An existing dog could have barked; remove it from heap.
138             mh.push(t+k) # Reenter (if popped above) or add a new dog to the heap, with time first available for
barking!

```

```
139     print(mh.size())
140
141
142 def koffinhradi():
143
144     # Read in the tasks and sort them based on deadlines, and times they take.
145     tasks = []
146     n = int(input())
147     for _ in range(n):
148         t, s = map(int, input().split())
149         tasks.append((s, t))
150     tasks.sort()
151     # print(tasks)
152
153     # We use the PQ to keep track of the tasks we have seen so far.
154     # We use the heap as a "max-pq", by inserting -t, such that the most
155     # time-consuming task is on top.
156     mh = PQMinHeap()
157     tt, d = 0, 0
158     for s, t in tasks:
159         mh.push(-t)
160         tt += t
161         while tt > s:
162             max_t = -mh.pop()
163             half_time = max_t // 2
164             mh.push(-half_time)
165             tt -= max_t
166             tt += half_time
167             d += 1
168     print(d)
169
170
```